# Introduction to Errors in Java

An error is a problem in a program that prevents it from compiling, executing correctly, or producing the expected result.

Errors are commonly classified into three main categories:

1. Compile-time errors
2. Runtime errors
3. Logical errors

## Compile-time errors

Compile-time errors occur when the source code violates Java's syntax or compile-time rules. The compiler reports these errors, and the program cannot be compiled successfully until they are corrected.

Example:

```
class Main {
    public static void main(String[] args) {
        int a = 10
        System.out.println(a);
    }
}
```

Error: A semicolon is missing after `int a = 10`.

Correction:

```
int a = 10;
```

Other examples include misspelled keywords, incompatible types, and using an undeclared variable.

## Runtime errors

Runtime errors occur while a program is executing. They may cause abnormal termination or an exception if the situation is not handled.

Example:

```
class Main {
    public static void main(String[] args) {
        int a = 10;
        int b = 0;

        System.out.println(a / b);
    }
}
```

Result: The program throws `ArithmeticException` because an integer is divided by zero.

Other examples include accessing an array with an invalid index and attempting to use a null reference.

### Logical errors

Logical errors occur when a program compiles and executes but produces an incorrect result because its logic or formula is wrong.

Example:

```
class Main {
    public static void main(String[] args) {
        int length = 10;
        int breadth = 5;

        int area = 2 * (length + breadth);
        System.out.println(area);
    }
}
```

Output:

```
30
```

The program calculates the perimeter of a rectangle, not its area. The correct formula for area is:

```
int area = length * breadth;
```

### Difference between the three types of errors

| Compile-time error | Runtime error | Logical error |
|---|---|---|
| Detected during compilation | Occurs during execution | Discovered by checking the result |
| Program cannot compile successfully | Program compiles, but execution may fail | Program may run normally |
| Example: missing semicolon | Example: division by zero | Example: incorrect formula |

# Introduction to Exceptions in Java

An exception is an event that occurs during program execution and disrupts the normal flow of instructions.

Java provides an exception-handling mechanism to detect and handle exceptional situations so that a program can respond appropriately.

Examples of exceptions:

- `ArithmeticException` — illegal integer arithmetic, such as division by zero.
- `ArrayIndexOutOfBoundsException` — accessing an array with an invalid index.
- `NullPointerException` — attempting certain operations through a null reference.
- `NumberFormatException` — attempting to convert an invalid numeric string into a number.
- `IOException` — an input/output operation fails.

### Exception hierarchy

The main hierarchy is:

Object

Throwable

Error

Serious problems, such as `OutOfMemoryError`

Exception

Conditions applications may handle

In Java, `Error` and `Exception` are both subclasses of `Throwable`.

Important: Not every runtime problem is an exception. For example, a logical error is not automatically represented by an exception.

## Types of exceptions

Java exceptions are commonly divided into two categories.

## A. Checked exceptions

Checked exceptions are checked by the compiler. A method must handle them with `try-catch` or declare them with `throws`, unless another permitted handling mechanism applies.

Examples:

- `IOException`
- `FileNotFoundException`
- `SQLException`
- `ClassNotFoundException`

## B. Unchecked exceptions

Unchecked exceptions are subclasses of `RuntimeException`. The compiler does not require a method to catch or declare them.

Examples:

- `ArithmeticException`
- `NullPointerException`
- `ArrayIndexOutOfBoundsException`
- `NumberFormatException`

| Checked exceptions | Unchecked exceptions |
|---|---|
| Checked by the compiler | Not required to be caught or declared |
| Often related to external operations | Often related to programming mistakes or invalid operations |
| Example: `IOException` | Example: `NullPointerException` |

`Error` objects are also unchecked in Java's type system, but they are distinct from exceptions and usually represent serious JVM or environment problems.

# Exception Handling in Java

Exception handling is a mechanism used to detect and respond to exceptions so that the program can deal with exceptional conditions instead of allowing them to cause an uncontrolled failure.

Java provides five important keywords:

| Keyword | Purpose |
|---|---|
| `try` | Encloses code that may throw an exception |
| `catch` | Handles a matching exception |
| `finally` | Runs after `try`/`catch` completes, subject to abnormal JVM termination |
| `throw` | Explicitly throws an exception object |
| `throws` | Declares exceptions a method may propagate |

## General syntax of exception handling

```
try {
    // Code that may throw an exception
}
catch (ExceptionType1 e) {
    // Handle the first type of exception
}
catch (ExceptionType2 e) {
    // Handle the second type of exception
}
finally {
    // Cleanup code, if required
}
```

Explanation:

1. The `try` block contains potentially exceptional code.
2. If an exception occurs, Java searches for a matching `catch` block.
3. The matching handler executes.
4. The `finally` block generally executes whether an exception occurs or not, except in cases such as abrupt JVM termination.

A `try` block must be followed by at least one `catch` block, a `finally` block, or both.

## Example of try-catch

```
class Main {
    public static void main(String[] args) {
        try {
            int a = 10;
            int b = 0;

            System.out.println(a / b);
        }
        catch (ArithmeticException e) {
            System.out.println("Cannot divide by zero.");
        }

        System.out.println("Program continues.");
```

```
    }
}
```

Output:

```
Cannot divide by zero.
Program continues.
```

Explanation: Division by zero throws `ArithmeticException`. The matching `catch` block handles it, and execution continues after the exception-handling statement.

# Multiple Catch Statements in Java

Multiple catch statements allow a single `try` block to be followed by several `catch` blocks, each designed to handle a different type of exception.

This is useful when different exceptional conditions require different responses.

## Syntax

```
try {
    // Statements that may throw different exceptions
}
catch (ArithmeticException e) {
    // Handle arithmetic exception
}
catch (ArrayIndexOutOfBoundsException e) {
    // Handle invalid array index
}
catch (NumberFormatException e) {
    // Handle invalid number conversion
}
```

## Example of multiple catch statements

```
class Main {
    public static void main(String[] args) {
        try {
            int[] arr = {10, 20, 30};

            int result = 10 / 2;
            System.out.println("Result: " + result);

            System.out.println(arr[5]);
        }
        catch (ArithmeticException e) {
            System.out.println("Arithmetic error occurred.");
        }
        catch (ArrayIndexOutOfBoundsException e) {
            System.out.println("Invalid array index.");
        }
        catch (Exception e) {
            System.out.println("Some other exception occurred.");
        }

        System.out.println("End of program.");
    }
}
```

Output:

```
Result: 5
Invalid array index.
End of program.
```

Explanation:

- The division `10 / 2` executes successfully.
- `arr[5]` throws `ArrayIndexOutOfBoundsException` because the valid indexes are `0`, `1`, and `2`.
- The second `catch` block handles the exception.
- The final general `catch (Exception e)` is not executed because the exception has already been handled.

## Rules for multiple catch blocks

1. A single `try` block can have multiple `catch` blocks.
2. Only the first matching `catch` block is executed for a given exception.
3. Specific exception types should appear before their more general superclass types.
4. A general `catch (Exception e)` should normally appear last.
5. Unreachable catch blocks are compile-time errors. For example, placing `catch (Exception e)` before `catch (ArithmeticException e)` makes the latter unreachable.
6. If an exception does not match any catch block, it propagates to the calling method.

## Multi-catch statement

Java also allows multiple exception types in one catch block, introduced in Java 7.

```
try {
    int n = Integer.parseInt("abc");
}
catch (NumberFormatException | NullPointerException e) {
    System.out.println("Invalid input.");
}
```

This is called multi-catch. It is different from writing multiple separate `catch` blocks.

## The finally Block

The `finally` block contains code that generally runs after the `try` and any matching `catch` block. It is often used for cleanup.

## Example

```
class Main {
    public static void main(String[] args) {
        try {
            int a = 10 / 0;
        }
        catch (ArithmeticException e) {
            System.out.println("Exception handled.");
        }
        finally {
            System.out.println("Finally block executed.");
        }
    }
}
```

Output:

```
Exception handled.
Finally block executed.
```

**The `finally` block generally executes even if an exception occurs or a `return` statement is encountered. It is not guaranteed to execute if the JVM terminates abruptly.**

### Difference Between throw and throws

| `throw` | `throws` |
|---|---|
| Used to explicitly throw an exception object | Used in a method declaration to specify exceptions that may be propagated |
| Used inside a method or block | Used in a method or constructor signature |
| Throws one exception object at a time | Can declare multiple exception types |
| Example: `throw new ArithmeticException();` | Example: `void read() throws IOException` |

### Example of `throw`

```
class Main {
    public static void main(String[] args) {
        int age = 15;

        if (age < 18) {
            throw new IllegalArgumentException("Age must be 18 or above");
        }
    }
}
```

This program throws an unchecked exception when `age` is less than 18. Since it is not caught, it propagates and normally terminates the program with a stack trace.

### Advantages of Exception Handling

1. Prevents many exceptional situations from causing uncontrolled program failure.
2. Separates error-handling code from normal program logic.
3. Allows different exceptions to be handled differently.
4. Supports propagation of exceptions through method calls.
5. Helps maintain program readability and reliability.

# Using `finally` Statements in Java

The `finally` block in Java is used to execute important cleanup code after a `try` block and its associated `catch` blocks. It generally executes whether an exception occurs or not.

It is commonly used for releasing resources, closing files, and performing cleanup operations.

### Syntax

```
try {
    // Code that may cause an exception
}
catch (ExceptionType e) {
    // Exception-handling code
}
finally {
```

```
    // Cleanup code
}
```

The `finally` block can be used with a `try-catch` statement or with `try` alone.

## Example: finally with an exception

```
class Main {
    public static void main(String[] args) {
        try {
            int a = 10 / 0;
        }
        catch (ArithmeticException e) {
            System.out.println("Exception handled");
        }
        finally {
            System.out.println("Finally block executed");
        }

        System.out.println("Program completed");
    }
}
```

Output:

```
Exception handled
Finally block executed
Program completed
```

Explanation:

- The `try` block throws an `ArithmeticException`.
- The `catch` block handles the exception.
- The `finally` block executes after the handler.
- The program continues with the next statement.

## Example: finally without an exception

```
class Main {
    public static void main(String[] args) {
        try {
            System.out.println("Inside try block");
        }
        finally {
            System.out.println("Inside finally block");
        }
    }
}
```

Output:

```
Inside try block
Inside finally block
```

## Uses of the finally block

1. Closing files after reading or writing.
2. Releasing resources.
3. Closing database connections when appropriate.
4. Performing cleanup operations.

5. Executing code that should run after normal exception handling.

Important note: The `finally` block is not guaranteed to execute if the JVM terminates abruptly, such as through `System.exit()` or a process crash. For resources implementing `AutoCloseable`, Java's try-with-resources statement is generally preferred because it closes resources automatically.

# Throwing Our Own Exceptions in Java

Java allows programmers to create and throw exceptions when a specific condition in their application is invalid. These are often called user-defined exceptions or custom exceptions.

A programmer can use the `throw` keyword to throw an exception explicitly. A custom exception class is generally created by extending `Exception` or `RuntimeException`.

## The throw keyword

The `throw` keyword is used to throw an exception object explicitly.

Syntax:

```
throw new ExceptionClass("Error message");
```

Example:

```
throw new ArithmeticException("Invalid arithmetic operation");
```

This statement creates and throws an `ArithmeticException` object.

## Example: Throwing a built-in exception

```
class Main {
    public static void main(String[] args) {
        int age = 15;

        try {
            if (age < 18) {
                throw new IllegalArgumentException(
                    "Age must be 18 or above"
                );
            }

            System.out.println("Eligible");
        }
        catch (IllegalArgumentException e) {
            System.out.println(e.getMessage());
        }
    }
}
```

Output:

```
Age must be 18 or above
```

Explanation: When `age` is less than 18, the program explicitly throws an `IllegalArgumentException`. The matching `catch` block handles it.

## Creating a user-defined exception

A user-defined exception is a class created by the programmer to represent a specific application error.

## Example: Invalid age exception

```
class InvalidAgeException extends Exception {
    InvalidAgeException(String message) {
        super(message);
    }
}

class Main {
    public static void main(String[] args) {
        int age = 16;

        try {
            if (age < 18) {
                throw new InvalidAgeException(
                    "Age is below 18"
                );
            }

            System.out.println("Valid age");
        }
        catch (InvalidAgeException e) {
            System.out.println(e.getMessage());
        }
    }
}
```

Output:

```
Age is below 18
```

## Explanation of the program

1. `InvalidAgeException` is a custom checked exception because it extends `Exception`.
2. Its constructor calls `super(message)` to pass the message to the superclass.
3. The `if` statement checks whether the age is below 18.
4. If the condition is true, `throw` creates and throws the custom exception.
5. The `catch` block handles the exception and prints its message.

Because this is a checked exception, code that throws it must catch it or declare it with `throws`.

## Checked versus unchecked custom exceptions

| Checked custom exception | Unchecked custom exception |
|---|---|
| Extends `Exception` directly or through a subclass other than `RuntimeException` | Extends `RuntimeException` |
| Must be caught or declared | Need not be caught or declared |
| Suitable when callers are expected to handle a recoverable condition | Suitable for invalid operations or programming/API contract violations |
| Example: `class MyException extends Exception` | Example: `class MyException extends RuntimeException` |

### Example of an unchecked custom exception

```
class InvalidMarksException extends RuntimeException {
    InvalidMarksException(String message) {
        super(message);
    }
}
```

This exception can be thrown without requiring the calling method to catch it or declare it.

### Advantages of user-defined exceptions

- Represent application-specific errors clearly.
- Make error messages more meaningful.
- Improve code readability.
- Allow different error conditions to be handled separately.
- Help separate validation rules from normal program logic.

## Using Exceptions for Debugging

Debugging is the process of identifying, analyzing, and correcting errors in a program.

Exceptions help with debugging because they can reveal where an operation failed, what type of problem occurred, and which method calls led to the failure.

### How exceptions help in debugging

1. Exception type: Identifies the category of failure, such as `NullPointerException`.
2. Exception message: Provides information about the failure.
3. Stack trace: Shows the sequence of method calls leading to the exception.
4. Targeted catch blocks: Allow a programmer to inspect and handle particular errors.
5. Logging: Records exception details to help diagnose problems later.

### Example: Printing an exception message

```
class Main {
    public static void main(String[] args) {
        try {
            int[] numbers = {10, 20, 30};
            System.out.println(numbers[5]);
        }
        catch (ArrayIndexOutOfBoundsException e) {
            System.out.println("Error: " + e.getMessage());
        }
    }
}
```

Output: The program prints an error message describing the invalid array index. The exact wording can vary by Java implementation.

Explanation: The array has valid indexes `0`, `1`, and `2`. Accessing index `5` throws `ArrayIndexOutOfBoundsException`. The exception message helps identify the problem.

### Using `printStackTrace()`

The `printStackTrace()` method prints details of an exception, including its type and the sequence of method calls associated with it.

```
class Main {
    public static void main(String[] args) {
        try {
            int result = 10 / 0;
        }
        catch (ArithmeticException e) {
            System.out.println("An error occurred:");
            e.printStackTrace();
        }
    }
}
```

Output: The output includes a message similar to:

```
An error occurred:
java.lang.ArithmeticException: / by zero
    at Main.main(Main.java:...)
```

The exact file name, line number, and formatting depend on the source file and runtime environment. Stack traces are commonly written to the standard error stream.

## Using `getMessage()` and `toString()`

| Method | Purpose |
|---|---|
| `getMessage()` | Returns the exception's detail message, or `null` if there is none |
| `toString()` | Returns a string containing the exception class name and, if present, its message |
| `printStackTrace()` | Prints exception details and stack-trace information |

## Example

```
class Main {
    public static void main(String[] args) {
        try {
            throw new Exception("Something went wrong");
        }
        catch (Exception e) {
            System.out.println(e.getMessage());
            System.out.println(e.toString());
        }
    }
}
```

Output:

```
Something went wrong
java.lang.Exception: Something went wrong
```

## Best practices when debugging exceptions

- Catch specific exceptions when you know how to handle them.
- Avoid empty `catch` blocks because they hide failures.
- Record useful context, such as the operation being performed.
- Use stack traces during development to locate the source of an error.
- Do not use exceptions as a substitute for ordinary control flow.
- Avoid exposing sensitive information in error messages shown to end users.

### Difference Between `finally`, `throw`, and `throws`

| `finally` | `throw` | `throws` |
|---|---|---|
| Defines a cleanup block | Explicitly throws an exception object | Declares exceptions a method may propagate |
| Used with `try` | Used inside executable code | Used in a method or constructor declaration |
| Generally executes after `try`/`catch` | Transfers control to exception handling or propagation | Informs callers about declared exceptions |
| Example: `finally { cleanup(); }` | `throw new Exception("Error");` | `void read() throws IOException` |

### Quick revision

- `finally`: Used for cleanup after exception handling.
- `throw`: Explicitly throws an exception object.
- Custom exception: A programmer-defined exception class.
- `extends Exception`: Usually creates a checked custom exception.
- `extends RuntimeException`: Creates an unchecked custom exception.
- `getMessage()`: Retrieves the exception's detail message.
- `printStackTrace()`: Prints exception and call-stack details.

# Introduction to Multithreaded Programming

Multithreading is a feature of Java that allows two or more threads to execute concurrently within a single program. A thread is a lightweight unit of execution.

A Java program can perform different tasks using separate threads, such as downloading a file while updating a progress indicator.

### Example

A word processor may use different threads for:

- Accepting user input.
- Checking spelling.
- Saving a document.
- Updating the display.

### Advantages of multithreading

1. Responsiveness: A program can remain responsive while another task is running.
2. Resource sharing: Threads in the same process share memory and other process resources.
3. Concurrency: Multiple tasks can make progress during overlapping periods.
4. Efficiency: Threads can be less costly to create and manage than separate processes.
5. Parallelism: On a multicore processor, threads may run simultaneously.

### Process versus thread

| Process | Thread |
|---|---|
| An executing program with its own process resources | A unit of execution within a process |

| Has its own process address space | Shares process memory with other threads |
|---|---|
| Generally more expensive to create | Generally less expensive to create |
| Processes are isolated from one another by default | Threads can access shared objects, so synchronization may be needed |

Important: Concurrency means tasks make progress over overlapping periods. Parallelism means tasks actually execute at the same time.

# Creating Threads in Java

Java commonly provides two basic approaches to creating a task that runs on a thread:

1. Extending the `Thread` class.
2. Implementing the `Runnable` interface.

The thread is started by calling `start()`, which arranges for the JVM to execute its `run()` method on a new thread.

### Method 1: Extending the Thread class

### Syntax

```
class MyThread extends Thread {
    public void run() {
        // Statements executed by the thread
    }
}

class Main {
    public static void main(String[] args) {
        MyThread t = new MyThread();
        t.start();
    }
}
```

### Example

```
class MyThread extends Thread {
    public void run() {
        for (int i = 1; i <= 5; i++) {
            System.out.println("Child thread: " + i);
        }
    }
}

class Main {
    public static void main(String[] args) {
        MyThread t = new MyThread();
        t.start();

        System.out.println("Main thread");
    }
}
```

Possible output:

```
Main thread
Child thread: 1
Child thread: 2
```

```
Child thread: 3
Child thread: 4
Child thread: 5
```

The exact output order is not guaranteed because the main thread and child thread execute concurrently.

### Method 2: Implementing the Runnable interface

The `Runnable` interface represents a task that can be executed by a thread.

```
class MyTask implements Runnable {
    public void run() {
        System.out.println("Runnable task is running");
    }
}

class Main {
    public static void main(String[] args) {
        MyTask task = new MyTask();
        Thread t = new Thread(task);
        t.start();
    }
}
```

Output:

```
Runnable task is running
```

### Difference between the two methods

| Extending `Thread` | Implementing `Runnable` |
|---|---|
| Class extends `Thread` | Class implements `Runnable` |
| Cannot extend another class at the same time | Can extend another class |
| Task and thread are combined in one class | Separates the task from the thread |
| Suitable for simple examples | Often more flexible and reusable |

Exam point: Calling `run()` directly does not start a new thread. Calling `start()` starts a new thread, which then invokes `run()`.

# Stopping and Blocking a Thread

Your topic “scopping” is likely referring to stopping a thread.

### 1 Stopping a thread

A thread stops executing when its `run()` method completes or exits because of an uncaught exception.

The older methods `stop()`, `suspend()`, and `resume()` are deprecated because they can cause unsafe behavior, such as inconsistent shared data or deadlocks.

A safer approach is to use a flag to request that a thread finish its work.

**Example: Stopping a thread using a flag**

```
class MyThread extends Thread {
```

```
    private volatile boolean running = true;

    public void run() {
        int i = 1;

        while (running && i <= 5) {
            System.out.println(i++);
        }
    }

    public void requestStop() {
        running = false;
    }
}

class Main {
    public static void main(String[] args)
            throws InterruptedException {
        MyThread t = new MyThread();
        t.start();

        Thread.sleep(100);
        t.requestStop();
    }
}
```

This example requests that the thread stop. Because thread scheduling is nondeterministic, the number of printed values can vary.

The `volatile` keyword ensures that updates to `running` are visible between threads. For more complex cancellation, Java commonly uses interruption or concurrency utilities.

## 2 Blocking a thread

Blocking means a thread temporarily cannot make progress because it is waiting for an event, a resource, or a particular condition.

Common situations include:

- Calling `Thread.sleep()` to pause for a specified duration.
- Calling `join()` to wait for another thread to finish.
- Calling `wait()` to wait for a notification while releasing the object's monitor.
- Waiting to acquire a lock held by another thread.
- Waiting for input/output operations to complete.

**Example: Using `sleep()`**

```
class Main {
    public static void main(String[] args)
            throws InterruptedException {
        for (int i = 1; i <= 3; i++) {
            System.out.println(i);
            Thread.sleep(1000);
        }
    }
}
```

Output: The program prints `1`, `2`, and `3`, approximately one second apart.

`Thread.sleep()` pauses the current thread, but it does not release any monitor locks that the thread already holds.

# Life Cycle of a Thread

A Java thread passes through different states during its lifetime.

## 1. Thread life-cycle diagram

NEW

Thread object created

RUNNABLE

Eligible to execute or running

May transition to waiting or timed waiting

BLOCKED

Waiting for a monitor lock

WAITING

Waiting indefinitely for an event

TIMED_WAITING

Waiting for a limited time

After the relevant event, a thread may return to RUNNABLE.

TERMINATED

Execution has completed

## 2. Explanation of thread states

Java officially defines six states through `Thread.State`.

| State | Meaning |
|---|---|
| `NEW` | The thread object has been created, but `start()` has not been called |
| `RUNNABLE` | The thread is eligible to run or is running; Java combines these conditions in one state |
| `BLOCKED` | The thread is waiting to acquire a monitor lock |
| `WAITING` | The thread is waiting indefinitely for another action |
| `TIMED_WAITING` | The thread is waiting for a specified period |
| `TERMINATED` | The thread has finished execution |

Important: “Running” is not a separate state in Java's official `Thread.State` enumeration.

# Using Thread Methods

Java's `Thread` class provides several methods to control and inspect threads.

### Important methods

| Method | Description |
|---|---|
| `start()` | Starts a new thread of execution |
| `run()` | Contains the task performed by the thread |
| `sleep(ms)` | Pauses the current thread for a specified time |
| `join()` | Waits for another thread to finish |
| `isAlive()` | Checks whether a thread has started and has not terminated |
| `currentThread()` | Returns the currently executing thread |
| `getName()` | Returns the thread's name |
| `setName(name)` | Changes the thread's name |
| `getPriority()` | Returns the thread's priority |
| `setPriority(p)` | Sets the thread's priority |
| `interrupt()` | Requests interruption of a thread |
| `isInterrupted()` | Checks a thread's interrupt status |

#### Example using join() and getName()

```
class MyThread extends Thread {
    public void run() {
        System.out.println(
            Thread.currentThread().getName() + " is running"
        );
    }
}

class Main {
    public static void main(String[] args)
            throws InterruptedException {
        MyThread t = new MyThread();
        t.setName("Child");

        t.start();
        t.join();

        System.out.println("Child thread completed");
    }
}
```

Output:

```
Child is running
Child thread completed
```

Explanation: The `join()` method makes the main thread wait until the child thread finishes. Therefore, the completion message is printed afterward.

# Thread Exceptions

Exceptions can occur during thread execution just as they can in ordinary Java code.

### InterruptedException

Methods such as `sleep()` and `join()` can throw `InterruptedException`. This happens when another thread interrupts the waiting thread.

### Example

```
class Main {
    public static void main(String[] args) {
        try {
            Thread.sleep(2000);
            System.out.println("Sleep completed");
        }
        catch (InterruptedException e) {
            System.out.println("Thread was interrupted");
        }
    }
}
```

Normal output:

```
Sleep completed
```

If the thread is interrupted while sleeping, it may instead print:

```
Thread was interrupted
```

### Handling exceptions inside a thread

An exception thrown by a thread's `run()` method should be handled within the thread if the thread needs to recover from it.

```
class MyThread extends Thread {
    public void run() {
        try {
            int result = 10 / 0;
            System.out.println(result);
        }
        catch (ArithmeticException e) {
            System.out.println("Exception handled in child thread");
        }
    }
}

class Main {
    public static void main(String[] args) {
        MyThread t = new MyThread();
        t.start();
    }
}
```

Output:

```
Exception handled in child thread
```

An exception that escapes `run()` can terminate that thread. A `try-catch` block in the main thread does not automatically catch exceptions thrown by a separate child thread.

### Uncaught exception handler

Java supports an `UncaughtExceptionHandler` for processing exceptions that escape a thread's execution.

```
class Main {
    public static void main(String[] args) {
```

```
        Thread t = new Thread(() -> {
            throw new RuntimeException("Unexpected failure");
        });

        t.setUncaughtExceptionHandler((thread, error) -> {
            System.out.println(
                thread.getName() + ": " + error.getMessage()
            );
        });

        t.start();
    }
}
```

Output:

```
Thread-0: Unexpected failure
```

The default thread name shown here is typical but may differ depending on the thread's name.

# Thread Priority in Java

Thread priority is a value associated with a thread that can influence the thread scheduler's decisions about which thread to execute.

Java priorities range from `1` to `10`.

## Constants used for thread priority

| Constant | Value | Meaning |
|---|---|---|
| `Thread.MIN_PRIORITY` | 1 | Minimum priority |
| `Thread.NORM_PRIORITY` | 5 | Normal priority |
| `Thread.MAX_PRIORITY` | 10 | Maximum priority |

## Example

```
class MyThread extends Thread {
    public void run() {
        System.out.println(
            getName() + " Priority: " + getPriority()
        );
    }
}

class Main {
    public static void main(String[] args) {
        MyThread t1 = new MyThread();
        MyThread t2 = new MyThread();

        t1.setName("Thread-1");
        t2.setName("Thread-2");

        t1.setPriority(Thread.MIN_PRIORITY);
        t2.setPriority(Thread.MAX_PRIORITY);

        t1.start();
        t2.start();
    }
}
```

Possible output:

```
Thread-1 Priority: 1
Thread-2 Priority: 10
```

The order of the output is not guaranteed.

### Important points

- A new thread normally inherits its priority from its parent thread, subject to the applicable thread-group maximum.
- `setPriority()` changes a thread's priority within the permitted range.
- A higher priority does not guarantee that a thread will execute first.
- Priority behavior may vary between JVM implementations and operating systems.

# Synchronization in Java

Synchronization is a mechanism used to control access to shared resources when multiple threads execute concurrently.

Without synchronization, two or more threads may update the same data simultaneously, causing a race condition and producing incorrect results.

Synchronization helps ensure that critical sections execute with the required mutual exclusion and memory visibility.

### Why synchronization is required

Suppose two threads update the same bank account balance. Both read the old balance before either update is completed. One update may overwrite the other, causing a lost update.

Synchronization helps prevent this problem by ensuring that only one thread at a time can execute a particular synchronized section guarded by the same monitor.

### Types of synchronization

1. Synchronized method: Locks the object's monitor for an instance method, or the class monitor for a static synchronized method.
2. Synchronized block: Locks a specified object's monitor for a particular section of code.

### Synchronized method

### Syntax

```
synchronized void methodName() {
    // Critical section
}
```

#### Example

```
class Counter {
    private int count = 0;

    synchronized void increment() {
```

```
        count++;
    }

    int getCount() {
        return count;
    }
}

class Main {
    public static void main(String[] args)
            throws InterruptedException {
        Counter c = new Counter();

        Thread t1 = new Thread(() -> {
            for (int i = 0; i < 1000; i++) {
                c.increment();
            }
        });

        Thread t2 = new Thread(() -> {
            for (int i = 0; i < 1000; i++) {
                c.increment();
            }
        });

        t1.start();
        t2.start();

        t1.join();
        t2.join();

        System.out.println("Count: " + c.getCount());
    }
}
```

Output:

```
Count: 2000
```

Explanation:

- Both threads share the same `Counter` object.
- Both call `increment()` 1,000 times.
- The `synchronized` keyword allows only one thread at a time to execute `increment()` on that particular object.
- The final count is `2000`.

## Synchronized block

A synchronized block allows synchronization of only the code that needs protection.

## Syntax

```
synchronized (object) {
    // Critical section
}
```

### Example

```
class Display {
    void show(String message) {
        synchronized (this) {
```

```
            for (int i = 0; i < 3; i++) {
                System.out.println(message);
            }
        }
    }
}

class Main {
    public static void main(String[] args) {
        Display d = new Display();

        Thread t1 = new Thread(() -> d.show("Hello"));
        Thread t2 = new Thread(() -> d.show("Java"));

        t1.start();
        t2.start();
    }
}
```

Possible output:

```
Hello
Hello
Hello
Java
Java
Java
```

The output may appear in the opposite order, but each call's three lines remain together because both threads synchronize on the same `Display` object.

## Advantages of synchronization

1. Prevents race conditions on properly guarded shared data.
2. Provides mutual exclusion for critical sections.
3. Establishes memory-visibility guarantees.
4. Helps maintain data consistency.
5. Supports safe coordination between threads.

## Disadvantages of synchronization

- May reduce performance because threads may have to wait.
- Excessive synchronization can reduce concurrency.
- Incorrect lock ordering can cause deadlocks.
- Poorly designed locking can make programs difficult to maintain.

Exam point: Synchronization should be applied carefully to shared mutable data. It is not necessary for every method or every variable.

## Difference Between sleep() and wait()

This is an important short-answer question for Java examinations.

| `sleep()` | `wait()` |
|---|---|
| Defined in the `Thread` class | Defined in the `Object` class |
| Pauses the current thread for a period | Waits for notification or another specified condition |
| Does not release monitor locks held by the thread | Releases the monitor on which `wait()` is called |

| Can be called without owning an object's monitor | Must be called while owning the relevant object's monitor |
|---|---|
| Throws `InterruptedException` | Throws `InterruptedException` |

Example:

```
synchronized (lock) {
    lock.wait();
}
```

The thread must own `lock`'s monitor before calling `wait()`. It releases that monitor while waiting and reacquires it before returning.

## Quick Revision Table

| Topic | Key point |
|---|---|
| Multithreading | Multiple threads make progress concurrently |
| `Thread` class | One way to create a thread |
| `Runnable` interface | Represents a task for a thread |
| `start()` | Starts a new thread of execution |
| `run()` | Contains the thread's task |
| Blocking | A thread waits for a resource or event |
| Thread life cycle | Six official Java thread states |
| `sleep()` | Pauses the current thread |
| `join()` | Waits for another thread to finish |
| Thread priority | Ranges from 1 to 10 |
| Synchronization | Controls access to shared resources |
| `synchronized` | Provides monitor-based mutual exclusion |
| `InterruptedException` | May occur when an interrupt reaches a waiting thread |